

Grund Shop — Stripe Integration, Full Walkthrough

From secret rotation to the webhook handler: everything covered, in order

Contents

1. Rotating a Secret in Vercel
2. NEXT_PUBLIC_ prefix: publishable vs secret keys
3. The Checkout Component (StripePayment.tsx)
4. Creating the PaymentIntent (OrderDetailsPage)
5. How the client_secret Connects Browser and Server
6. confirmPayment, the Redirect, and 3D Secure
7. convertToPlainObject — Why It's Needed
8. Creating an Event Destination (Webhook) in the Dashboard
9. pk_ vs sk_ — Where Each Is Actually Used
10. The Success Page — Verifying the Payment
11. The Webhook — What It Is and When It Runs
12. The Webhook Handler — Code Review
13. Where the stripe-signature Header Comes From

1. Rotating a Secret in Vercel

“Rotate” means replace the old secret with a brand-new one and stop using the old one. **Analogy:** changing the lock on your house — if a key might have been copied or seen by others, you get a new lock and new keys, and the old key becomes useless.

Vercel flagged `PAYPAL_APP_SECRET`, `NEXTAUTH_SECRET`, and `DATABASE_URL` as “Needs Attention” because they were saved as plain “Config” variables, which anyone with project access can read. The fix:

- Generate a new secret at the source (e.g. the PayPal developer dashboard), invalidating the old one.
 - Save the new value in Vercel and mark it “Secret” (sensitive), so it can't be read back once saved.
 - Redeploy so the app picks up the new value.
-

2. The `NEXT_PUBLIC_` Prefix: Publishable vs Secret Keys

`NEXT_PUBLIC_` doesn't mean “insecure” — it tells Next.js this value is allowed to be sent to the browser.

Analogy: the publishable key is like a shop's street address on the front door, meant to be seen by everyone. The secret key is the safe combination, and that one must stay hidden.

Variable	Prefix	Where used
<code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code> (pk_...)	Needs <code>NEXT_PUBLIC_</code>	Browser (loadStripe)
<code>STRIPE_SECRET_KEY</code> (sk_...)	Never add <code>NEXT_PUBLIC_</code>	Server only (new Stripe(...))

Browsers only expose env vars starting with `NEXT_PUBLIC_`. Removing that prefix from the publishable key breaks checkout — loadStripe would receive undefined. Vercel's warning here is a generic reminder (it flags any `NEXT_PUBLIC_` variable with “KEY” in the name), not a real problem for this one.

3. The Checkout Component (`StripePayment.tsx`)

Analogy: loadStripe is getting the card machine from the bank. `<Elements>` is setting that machine on the counter, loaded with this specific bill (the clientSecret). `PaymentElement` is the machine's keypad and screen. `confirmPayment` is pressing “pay.”

Key pieces and what each does:

- `loadStripe(pk_...)` — loads Stripe.js in the browser and returns a promise for the Stripe object. Should be called once, outside the component, not on every render.
- `<Elements stripe={stripePromise} options={{ clientSecret }}>` — provides that Stripe object and the clientSecret via React context to everything nested inside it.
- `useStripe()` and `useElements()` — both read from that same context. `useStripe()` is not created by `useElements()`; they're siblings, not one derived from the other.
- `<PaymentElement>` — a Stripe-hosted iframe. Card details go straight to Stripe and never touch your server.

- `confirmPayment({ elements, confirmParams: { return_url } })` — reads the card details from the iframe, confirms with Stripe, then redirects to `return_url`. It only returns an error if something fails (e.g. declined card); a successful payment ends in a redirect, not a returned value your code reads.

Bugs identified in the original component:

- `process.env.STRIPE_PUBLISHABLE_KEY` (missing `NEXT_PUBLIC_`) evaluates to undefined in the browser — must be `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`, matching the name used in Vercel.
- `StripeForm` was defined *inside* `StripePayment`, so it gets re-created on every render, risking form resets while typing. Move it outside the component.
- `loadStripe(...)` should be called once at module scope, not inside the component body, so it doesn't reload on every render.

4. Creating the PaymentIntent (OrderDetailsPage)

```
if (order.paymentMethod === 'Stripe' && !order.isPaid) {
  const stripe = new Stripe(process.env.STRIPE_SECRET_KEY as string);
  const paymentIntent = await stripe.paymentIntents.create({
    amount: Math.round(Number(order.totalPrice) * 100), // to cents
    currency: 'usd',
    metadata: { orderId: order.id },
  });
  client_secret = paymentIntent.client_secret;
}
```

This runs on the server, using `sk_`, to create the `PaymentIntent` (the record of “this order needs this amount, in this currency”). The `PaymentIntent` itself stays on Stripe's servers — only its `client_secret` is sent down to the browser as a prop.

Known limitation: this creates a new `PaymentIntent` on every page load, so refreshing the order page creates duplicates. A more complete version would save the intent ID on the order and reuse it if one already exists.

5. How the client_secret Connects Browser and Server

Analogy: the `clientSecret` is a claim ticket. Stripe doesn't need to know who you are up front — you hand over the ticket, and Stripe looks up the exact bill it belongs to.

The chain:

- Server creates the `PaymentIntent` with `sk_` and gets back a `client_secret`.
- That `client_secret` is passed down as a prop: `OrderDetailsPage` → `OrderDetailsTable` → `StripePayment` → `<Elements>`.
- `<Elements>` shares the Stripe object and `client_secret` via React context.
- When `<PaymentElement>` mounts, it sends that `client_secret` to Stripe's servers, which look up the matching `PaymentIntent` (amount, currency, metadata) and return what the form needs to render.
- The `client_secret`'s ID portion (e.g. `pi_3Abc...`) identifies which `PaymentIntent`; the secret portion proves the browser is allowed to act on it. Neither part *contains* the transaction data itself — it's a lookup key, not

a payload.

6. confirmPayment, the Redirect, and 3D Secure

On submit, `stripe.confirmPayment({ elements, confirmParams: { return_url } })` sends the card details from the iframe to Stripe. Behind the scenes, Stripe may run 3D Secure (3DS) — the extra check card networks and issuing banks use to confirm the real cardholder is paying.

Analogy: you hand your card to the cashier, who quietly calls your bank in the background. “Frictionless”: the bank recognizes the purchase as normal and silently approves it — you see nothing. “Challenge”: the bank isn't sure and asks for proof — an OTP, a PIN, or approval in your banking app, shown as a popup.

Either way, Stripe handles this automatically with no extra code from you. On success (frictionless or after a completed challenge), the browser is redirected to `return_url` with query params appended: `payment_intent`, `payment_intent_client_secret`, and `redirect_status`. On failure, `confirmPayment` resolves with an error object your code displays.

Important: because a challenge can send the user away to their bank and back, you can't assume the payment finished just because the page loaded. The `redirect_status` query param is not proof by itself — it can be edited by hand. Always re-verify on the server (see Section 10).

7. convertToPlainObject — Why It's Needed

```
export async function getOrderByById(orderId: string) {
  return convertToPlainObject(
    await prisma.order.findFirst({
      where: { id: orderId },
      include: {
        orderitems: true,
        user: { select: { name: true, email: true } },
      },
    }),
  );
}
```

```
export function convertToPlainObject<T>(data: T): T {
  return JSON.parse(JSON.stringify(data))
}
```

Analogy: the server-to-client boundary is a border crossing. Only plain, “declared” cargo (strings, numbers, plain objects, arrays) is allowed through. Some Prisma fields come back packed in special containers (class instances), which get rejected unless repacked into plain boxes first.

- **Decimal fields** (`itemsPrice`, `shippingPrice`, `taxPrice`, `totalPrice`) — Prisma returns these as `Decimal` class instances, not plain numbers.
- **DateTime fields** (`createdAt`, `paidAt`, `deliveredAt`) — Prisma returns real JavaScript `Date` objects.
- **Json fields** (`orderItems`, `shippingAddress`, `PaymentResult`) are already plain, but the whole object still fails to serialize if any other field on it isn't.

`JSON.stringify` turns `Decimal` into a string like `"19.99"` (via its `toJSON()` method) and `Date` into an ISO string. `JSON.parse` then turns that string back into plain JS values with no class instances left. This matters because `getOrderByById`'s result gets passed to `OrderDetailsTable`, which is (or contains) a Client Component — passing

the raw Prisma object across that boundary would throw a serialization error.

Note on the redundant `.toString()` call: since `order.totalPrice` is already a plain string by the time `convertToPlainObject` runs, a later `.toString()` call on it in `OrderDetailsPage` is a no-op at runtime — it just returns the same string. It's still worth keeping: it costs nothing, documents intent, and guards against future changes to the data path (TypeScript still types the field as `Decimal` even after the runtime conversion strips the class away).

8. Creating an Event Destination (Webhook) in the Dashboard

Steps in Workbench's Webhooks tab:

1. Go to the Webhooks tab in Workbench (dashboard.stripe.com/webhooks).
2. Click “Add destination” (or “Create an event destination”).
3. Event destination scope: “Your account” — “Connected accounts” is only for Stripe Connect marketplaces/platforms, not used here.
4. API version: leave the default unless code targets an older version specifically.
5. Events: avoid “All events.” Select only what's needed — `payment_intent.succeeded` and `payment_intent.payment_failed` for this checkout flow.
6. Destination type: Webhook (sends to an HTTPS endpoint), not AWS EventBridge or Azure Event Grid.
7. Endpoint URL: the publicly reachable HTTPS URL Stripe should POST to, e.g. <https://yourdomain.com/api/webhooks/stripe>.
8. Destination name: an internal label only (e.g. “Grund Shop Sandbox - Order Payments”) — Stripe doesn't use it functionally.
9. Create the destination. Stripe reveals a signing secret starting with `whsec_` — save it as `STRIPE_WEBHOOK_SECRET`, marked Secret (not Config) in Vercel.

Testing locally (Stripe can't reach localhost directly):

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

This forwards test events to your local server and prints a temporary `whsec_` to use in `.env.local`. Alternative: a tunneling tool like ngrok to give localhost a temporary public URL.

9. `pk_` vs `sk_` — Where Each Is Actually Used

Key	Used where	Used for
<code>pk_</code> (publishable)	Browser only (<code>StripePayment.tsx</code>)	<code>loadStripe(...)</code> creates the Stripe.js object passed into <code><Elements stripe={...}></code> , rendering the card form.
<code>sk_</code> (secret)	Server only	Creating the <code>PaymentIntent</code> (<code>OrderDetailsPage</code>), retrieving/verifying it (<code>Successpage</code>), and verifying webhook signatures.

`pk_` is used exactly once in this codebase, inside `loadStripe`. It never touches order creation, payment verification, or the webhook handler — all of those are `sk_`.

10. The Success Page — Verifying the Payment

Analogy: the checkout page is the card machine; the success page is the back-office receipt printer. It never talks to the customer's card — it double-checks the sale really happened first.

```
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY as string);

const Successpage = async (props: {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ payment_intent: string }>;
}) => {
  const { id } = await props.params;
  const { payment_intent: paymentIntentId } = await props.searchParams;

  const order = await getOrderById(id);
  if (!order) notFound();

  const paymentIntent = await stripe.paymentIntents.retrieve(paymentIntentId);

  if (paymentIntent.metadata.orderId === null ||
    paymentIntent.metadata.orderId !== order.id) {
    return notFound();
  }

  const isSuccess = paymentIntent.status === 'succeeded';
  if (!isSuccess) return redirect(`/order/${id}`);

  // render "Thanks for your purchase"
};
```

- searchParams just parses the query params Stripe appended to return_url — no Stripe call yet at that point.
- stripe.paymentIntents.retrieve(paymentIntentId) is the real check: sk_ asks Stripe's API directly for the current, authoritative status and metadata, rather than trusting the URL.
- The metadata.orderId check blocks someone pasting a valid PaymentIntent ID for a different order into this URL to fake success here.
- status === 'succeeded' is the actual proof of payment. redirect_status in the URL is not proof by itself — it can be edited by hand.
- pk_, client_secret, and Stripe.js are not used anywhere on this page — those belong to the checkout page only. This page is plain server-side Next.js.

11. The Webhook — What It Is and When It Runs

Analogy: the success page is the cashier telling the customer “looks like that went through!” The webhook is the bank separately sending a confirmation slip to the back office — it doesn't care whether the customer is still there, whether their browser crashed, or whether the redirect ever completed.

The webhook runs completely independently of the browser. The instant the subscribed event happens on Stripe's servers, Stripe makes a server-to-server HTTP POST directly to the registered endpoint. This can happen even if the customer's browser never makes it back to the success page (closed tab, dropped connection, failed redirect).

Why it's still needed given the success page also checks the PaymentIntent: the success page's check only runs if the browser gets back to the site. The webhook is the reliable source of truth for actually marking the order paid; the success page is just UI feedback for the customer who's still there.

Why localhost doesn't work directly: Stripe's servers need a public, internet-reachable HTTPS URL to POST to. localhost has no public route. This is about reachability, not a “dev vs prod” restriction — the Stripe CLI's local listener (or ngrok) solves it for local development.

12. The Webhook Handler — Code Review

```
export async function POST(req: NextRequest) {
  const event = await Stripe.webhooks.constructEvent(
    await req.text(),
    req.headers.get('stripe-signature') as string,
    process.env.STRIPE_WEBHOOK_SECRET as string
  );

  if (event.type === 'charge.succeeded') {
    const { object } = event.data;
    await updateOrderToPaid({
      orderId: object.metadata.orderId,
      paymentResult: {
        id: object.id,
        status: 'COMPLETED',
        email_address: object.billing_details.email!,
        pricePaid: (object.amount / 100).toFixed(),
        name: '',
      },
    });
    return NextResponse.json({ message: 'UpdateOrderToPaid was successful' });
  }

  return NextResponse.json({ message: 'event is not charged.succeeded' });
}
```

Issues found:

- **Event type mismatch (important):** checks `event.type === 'charge.succeeded'`, but the dashboard destination was configured to send `payment_intent.succeeded` / `payment_intent.payment_failed`. If so, this if-block never runs and real payments silently fall through to the else branch. The event checked in code must match what's selected in the dashboard.
- **No try/catch around constructEvent:** an invalid/missing signature throws, currently resulting in a generic 500. Stripe expects a proper 400 for bad signatures.
- **(object.amount / 100).toFixed()** rounds to a whole number with no decimals — 1999 cents becomes "20", not "19.99". Should be `.toFixed(2)`.
- **Metadata location depends on event type:** for `payment_intent.succeeded`, `event.data.object` is a `PaymentIntent` and `metadata.orderId` is set directly. For `charge.succeeded`, confirm in a real test event that metadata was actually copied onto the `Charge`.
- **No idempotency check:** Stripe retries webhook deliveries on slow/error responses, so the same event can arrive twice. `updateOrderToPaid` should check `isPaid` first and skip if already true.

Suggested signature-verification fix:

```
let event: Stripe.Event;
try {
  event = Stripe.webhooks.constructEvent(
    await req.text(),
    req.headers.get('stripe-signature') as string,
    process.env.STRIPE_WEBHOOK_SECRET as string
  );
} catch (err) {
  return NextResponse.json({ message: 'Invalid signature' }, { status: 400 });
}
```

13. Where the stripe-signature Header Comes From

Stripe attaches a header called Stripe-Signature to every webhook POST it sends. It's a cryptographic signature computed from the raw request body combined with the `whsec_` signing secret shown when the event destination was created.

Why it matters: anyone could POST a fake payload pretending to be Stripe (e.g. claiming a payment succeeded, to get an order marked paid for free). The signature proves the request genuinely came from Stripe and wasn't altered in transit.

`constructEvent(body, signature, secret)` recomputes the expected signature from the raw body and `whsec_`, then compares it to the header. A mismatch (wrong secret, tampered body, forged request) causes it to throw.

Implementation detail: `constructEvent` needs the raw, unparsed body (`await req.text()` is correct). Calling `req.json()` first and re-stringifying would change the exact bytes and break the signature check, since even whitespace differences invalidate it.